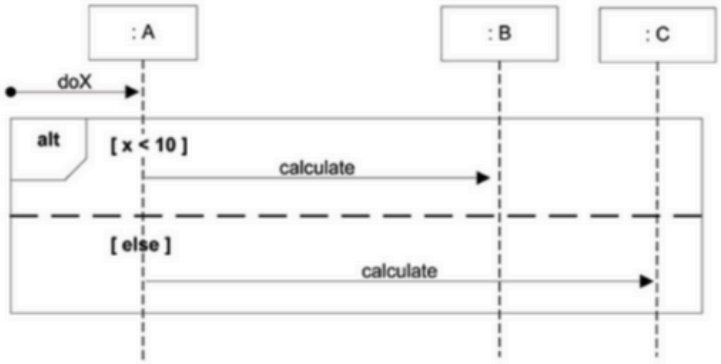
# Software Patterns - Problemsheet - 1

Implement the following sequence diagram in Java Programming Language.

1.




```
class B:
    def calculate(self):
        print("B.calculate()")


class C:
    def calculate(self):
        print("C.calculate()")

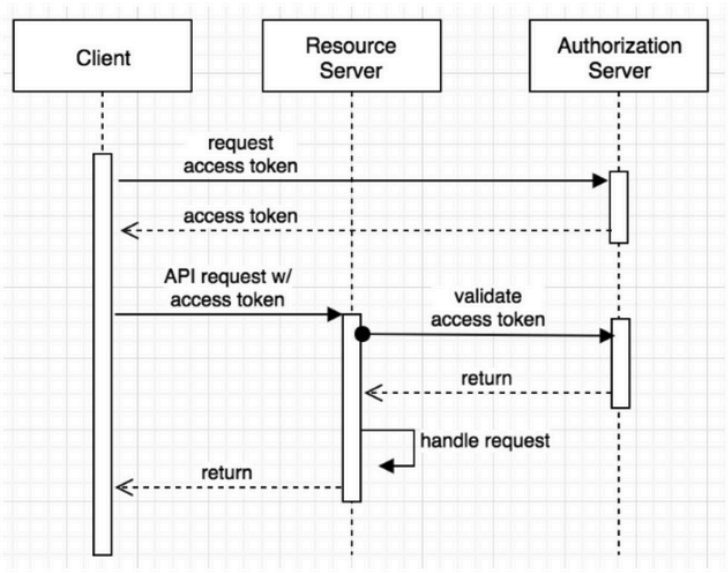

class A:
    def __init__(self, b, c):
        self.b = b
        self.c = c

    def doX(self, x):
        obj = self.b if x < 10 else self.c
        obj.calculate()


a = A(B(), C())
print("Testing with x = 5: ", end="")
a.doX(5)
print("Testing with x = 15: ", end="")
a.doX(15)
```

```
In [3]: runfile('D:/COLLEGE/SEM 9/Software Patterns/ps1_q1.py', wdir='D:/COLLEGE/SEM 9/Software Patterns')
Testing with x = 5: B.calculate()
Testing with x = 15: C.calculate()
```

2.




```
# Authorization Server Class
class AuthorizationServer:
    def request_access_token(self):
        print("AuthServer: Generating access token...")
        return "SECRET_TOKEN_123"  # Return access token

    def validate_access_token(self, token):
        print(f"AuthServer: Validating token: {token}")
        return token == "SECRET_TOKEN_123"  # Return validation result


# Resource Server Class
class ResourceServer:
    def __init__(self, auth_server):
        self.auth_server = auth_server

    def api_request(self, token):
        print("ResourceServer: Received API request.")

        # Validate access token (Call to Authorization Server)
        is_valid = self.auth_server.validate_access_token(token)

        if is_valid:
            self.handle_request()
            return "Success: Data Accessed"
```

```
        else:
            return "Error: Unauthorized"

    def handle_request(self):
        print("ResourceServer: Handling the request...")


# Client Class
class Client:
    def __init__(self, auth_server, resource_server):
        self.auth_server = auth_server
        self.resource_server = resource_server

    def start(self):
        # Request access token
        token = self.auth_server.request_access_token()
        print(f"Client: Received token: {token}")

        # API request with access token
        response = self.resource_server.api_request(token)
        print(f"Client: Final response: {response}")


# Main function
def main():
    auth_server = AuthorizationServer()
    resource_server = ResourceServer(auth_server)
    client = Client(auth_server, resource_server)

    client.start()


if __name__ == "__main__":
    main()
```

```
In [4]: runfile('D:/COLLEGE/SEM 9/Software Patterns/ps1_q2.py', wdir='D:/COLLEGE/SEM 9/Software Patterns')
AuthServer: Generating access token...
Client: Received token: SECRET_TOKEN_123
ResourceServer: Received API request.
AuthServer: Validating token: SECRET_TOKEN_123
ResourceServer: Handling the request...
Client: Final response: Success: Data Accessed
```

---

## Code to class diagram

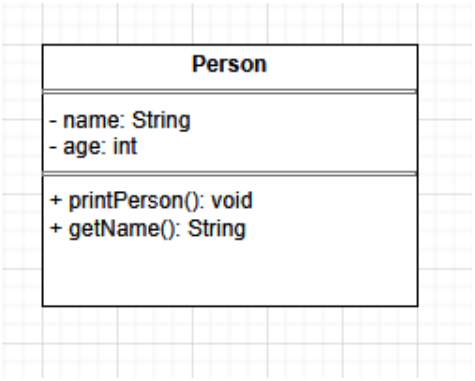
1. C++

```
public class Person {
private String name;
private int age;

public Person(String initialName) {
this.name = initialName;
this.age = 0;
}

public void printPerson() {
System.out.println(this.name + ", age " + this.age + " years");
}

public String getName() {
return this.name;
}
};
```



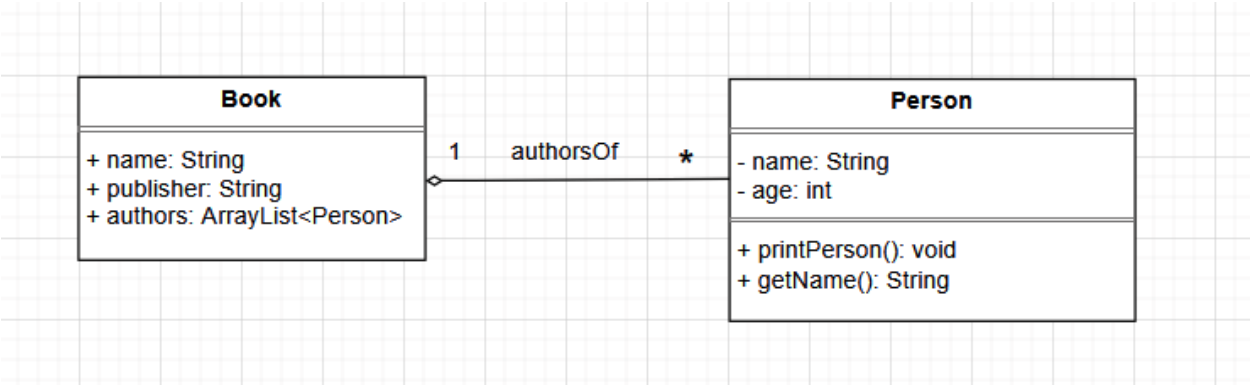

---

2. c++

```
public class Book {
private String name;

private String publisher;
private ArrayList<Person> authors;

// constructors and methods
};
```




---

3.java

```
public interface Number {
Number add(Number n);
Number subtract(Number n);
Number multiply(Number n);
Number divide(Number n);
}

public class RealNumber implements Number {

private double value;

public RealNumber(double value){ }

public Number add(Number n){ }

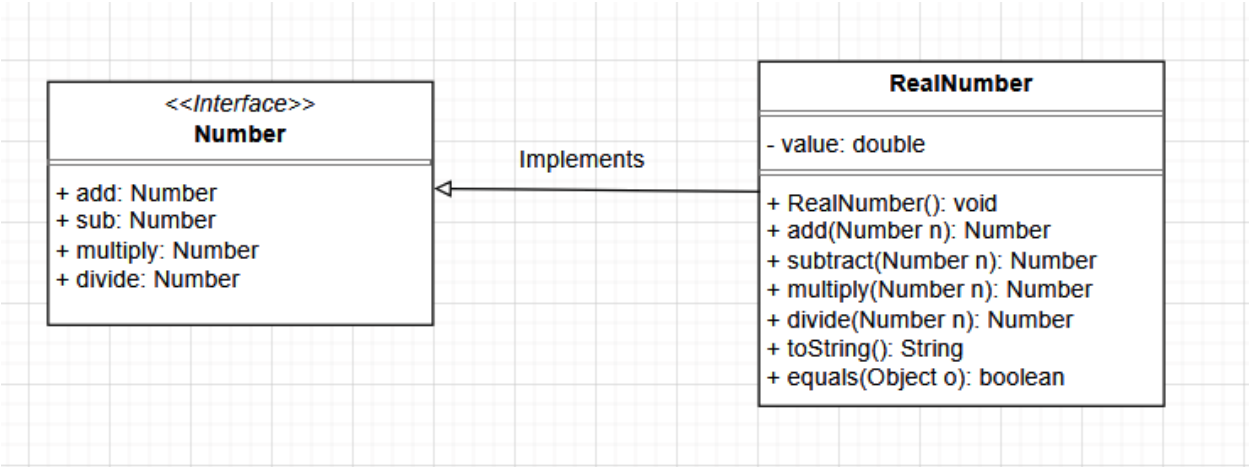
public Number subtract(Number n){ }

public Number multiply(Number n){ }

public Number divide(Number n){ }

@Override
public String toString(){ }

@Override
public boolean equals(Object o){ }
}
```



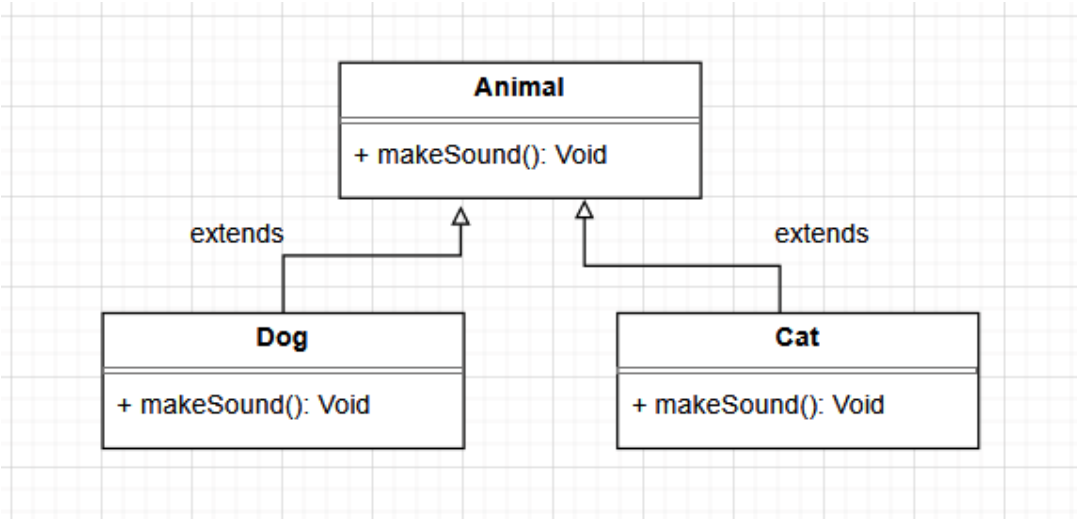

---

4. java

public class Animal { // Method that can be overridden public void makeSound() { System.out.println("Some generic animal sound"); } }

// Dog class (Subclass) public class Dog extends Animal { // Overriding the makeSound method @Override public void makeSound() { System.out.println("Bark"); } }

// Cat class (Subclass) public class Cat extends Animal { // Overriding the makeSound method @Override public void makeSound() { System.out.println("Meow"); } }




---

5. c++

```
// Engine class
class Engine {
public:

Engine() {
cout << "Engine Created!" << endl;
}

~Engine() {
cout << "Engine Destroyed!" << endl;
}

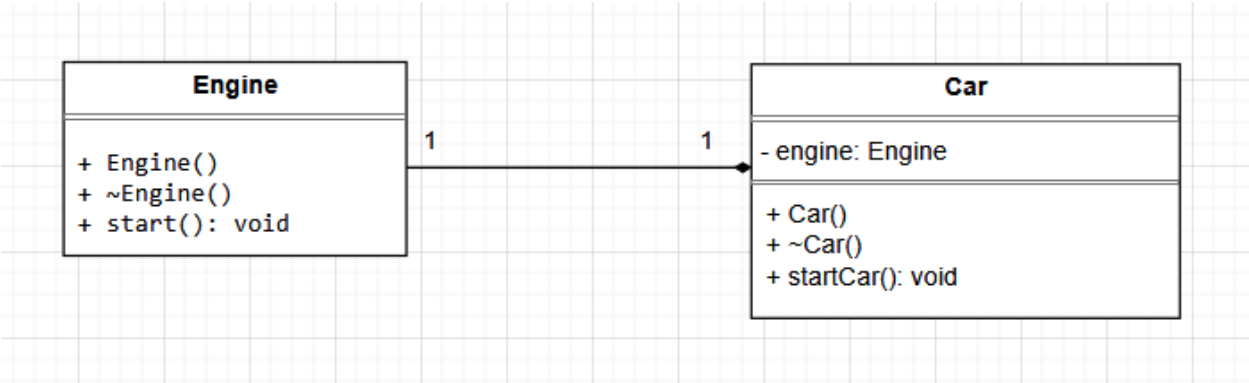
void start() {
cout << "Engine Started!" << endl;
}
};

// Car class containing an Engine (Composition)
class Car {
private:
Engine engine; // Car "has-a" Engine (Composition)

public:
Car() {
cout << "Car Created!" << endl;
}

~Car() {
cout << "Car Destroyed!" << endl;
}

void startCar() {
engine.start(); // Use the engine's start method
cout << "Car Started!" << endl;
}
};
```




---

6. // Employee class

```
class Employee {
private:
string name;
public:
Employee(string name) : name(name) {}

void showEmployeeInfo() {
cout << "Employee Name: " << name << endl;
}
};

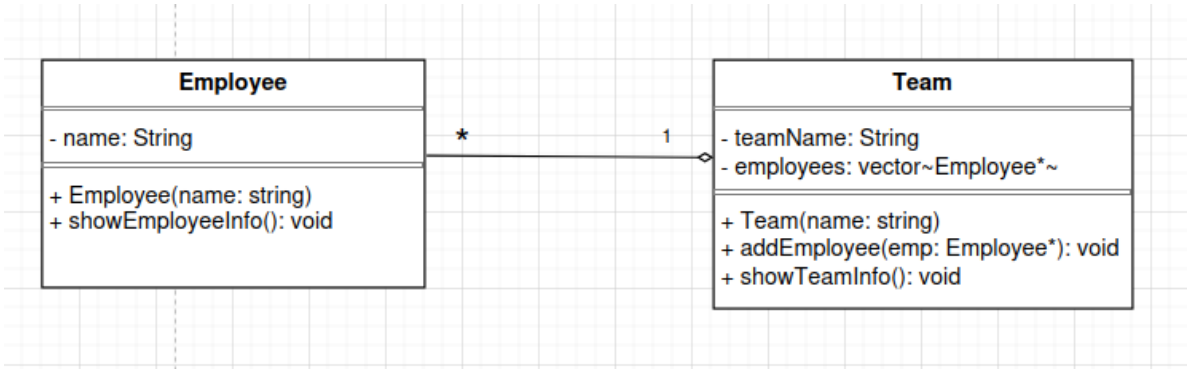
// Team class containing a list of Employees (Aggregation)
class Team {
private:
string teamName;
vector<Employee*> employees; // Team "has-a" list of Employees (Aggregation)

public:
Team(string name) : teamName(name) {}

// Method to add employee to the team
void addEmployee(Employee* emp) {
employees.push_back(emp);
}

void showTeamInfo() {
cout << "Team Name: " << teamName << endl;
for (auto& emp : employees) {
emp->showEmployeeInfo();
```

```
}
}
};
```




---

7.

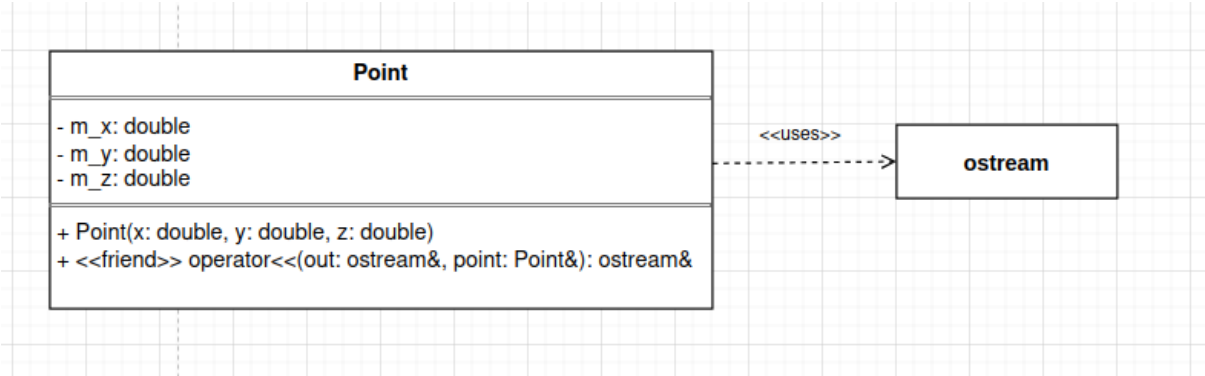
```
class Point
{
private:
double m_x{};
double m_y{};
double m_z{};

public:
Point(double x=0.0, double y=0.0, double z=0.0): m_x{x}, m_y{y}, m_z{z}
{
}

friend std::ostream& operator<< (std::ostream& out, const Point& point); // Point has a
dependency on std::ostream here
};

std::ostream& operator<< (std::ostream& out, const Point& point)
{
// Since operator<< is a friend of the Point class, we can access Point's members directly.
out << "Point(" << point.m_x << ", " << point.m_y << ", " << point.m_z << ')';

return out;
}
```




---

8.

```
// Student class
class Student {
private:
string name;

public:
Student(const string& name) : name(name) {}

string getName() const {
return name;
}
};

// Course class
class Course {
private:
string courseName;

public:
Course(const string& name) : courseName(name) {}

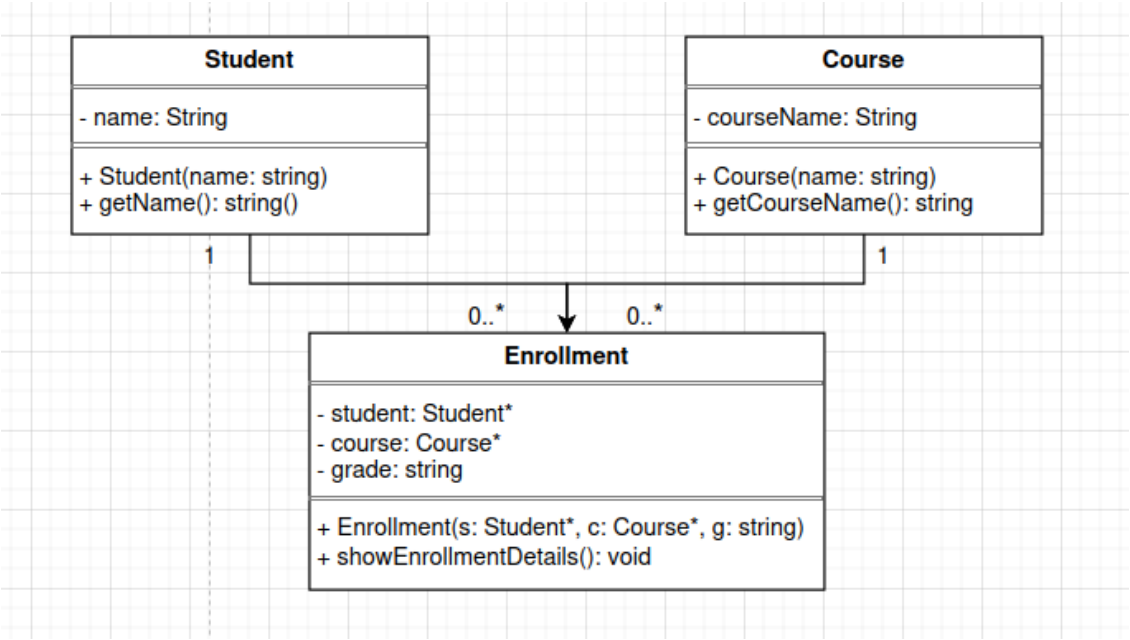
string getCourseName() const {
return courseName;
}
};

// Enrollment class (Association)
class Enrollment {
private:
```

```
Student* student;
Course* course;
string grade;

public:
// Constructor to associate a student with a course
Enrollment(Student* s, Course* c, const string& g)
: student(s), course(c), grade(g) {}

// Method to display enrollment details
void showEnrollmentDetails() const {
cout << student->getName() << " is enrolled in " << course->getCourseName()
<< " and received a grade of " << grade << "." << endl;
}
};
```




---

9.

```
class Machine {
protected:
std::string brand;

public:
// Constructor for Machine class
Machine(const std::string& brand) : brand(brand) {}
```

```
// A method that every machine can perform
void operate() {
std::cout << "The machine is operating." << std::endl;
}

// Getter for the brand
std::string getBrand() const {
return brand;
}
};
class Engine {
public:
// Method to start the engine
void start() {
std::cout << "Engine is starting..." << std::endl;
}

// Method to stop the engine
void stop() {
std::cout << "Engine is stopping..." << std::endl;
}
};
class Vehicle : public Machine {
private:
Engine engine; // Vehicle has an engine

public:
// Constructor for Vehicle class
Vehicle(const std::string& brand) : Machine(brand) {}

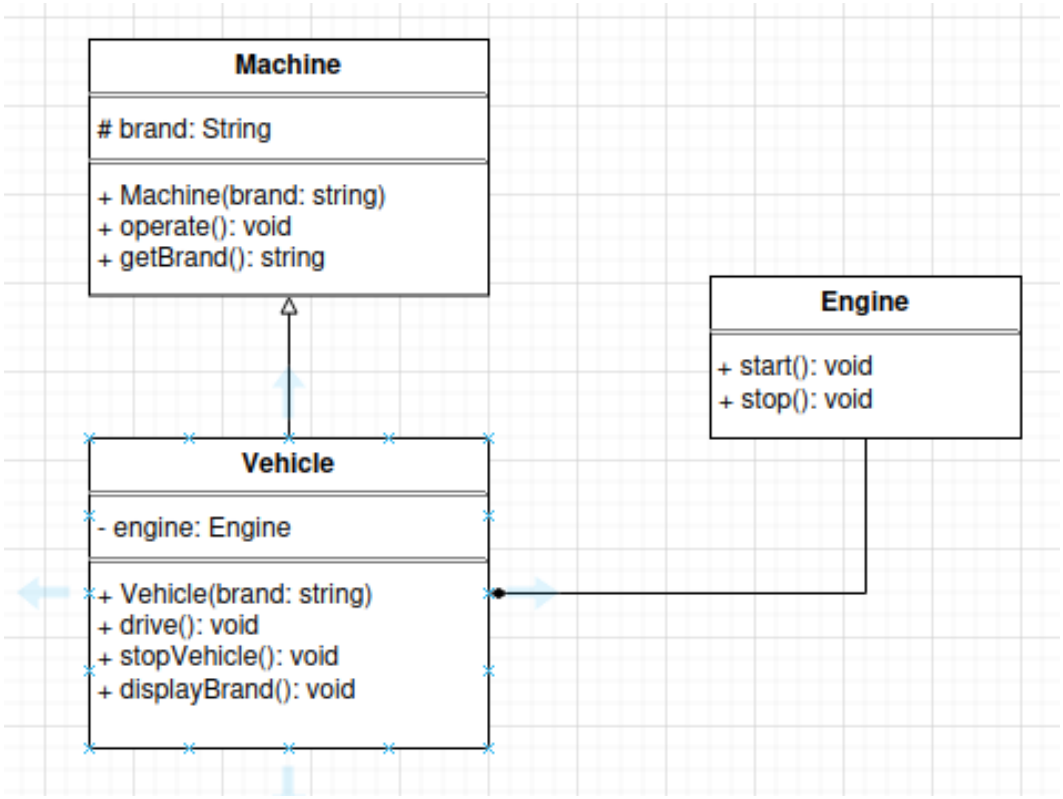
// Method to operate the vehicle

void drive() {
std::cout << "The vehicle is driving." << std::endl;
engine.start(); // Vehicle uses the engine to drive
}

// Method to stop the vehicle
void stopVehicle() {
std::cout << "The vehicle is stopping." << std::endl;
engine.stop(); // Vehicle uses the engine to stop
}

// Method to display the brand
```

```
void displayBrand() {
std::cout << "The vehicle's brand is: " << getBrand() << std::endl;
}
};
```




---

10.

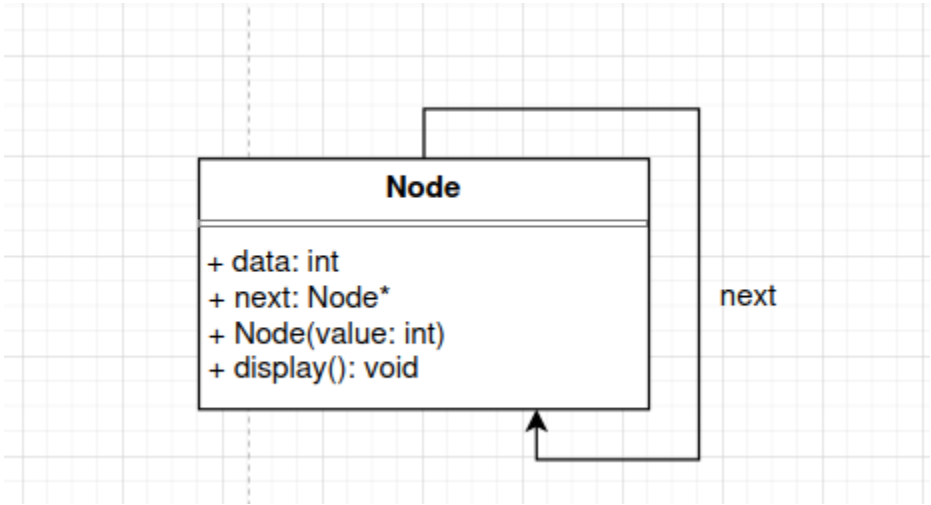
```
class Node {
public:
int data; // Data of the node
Node* next; // Pointer to the next node (self-referential)

// Constructor
Node(int value) : data(value), next(nullptr) {}

// Method to display the node data

void display() {
cout << "Node data: " << data << endl;
}
```

};


Node
+ data: int
+ next: Node*
+ Node(value: int)
+ display(): void
next